Software Requirements Specification

for

DAO-Based Ride Sharing Application

Version 1.1

Prepared by

Ram Sundar Radhakrishnan	**23BCE1939**	**ram.sundar2023@vitstudent.ac.in**

Instructor: **Berin Shalu S**

Course: **Software Engineering Lab BCSE301P**

Date: **16th December 2025**

Contents

CONTENTS ..... **II**

REVISIONS ..... **II**

1 INTRODUCTION **1**

1.1 DOCUMENT PURPOSE 1
1.2 PRODUCT SCOPE 1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW 1
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS 1
1.5 DOCUMENT CONVENTIONS 1
1.6 REFERENCES AND ACKNOWLEDGMENTS 1

2 OVERALL DESCRIPTION **3**

2.1 PRODUCT OVERVIEW 3
2.2 PRODUCT FUNCTIONALITY 4
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS 4
2.4 ASSUMPTIONS AND DEPENDENCIES 4

3 SPECIFIC REQUIREMENTS **5**

3.1 EXTERNAL INTERFACE REQUIREMENTS 5
3.2 FUNCTIONAL REQUIREMENTS 6
3.3 USE CASE MODEL 7

4 OTHER NON-FUNCTIONAL REQUIREMENTS **10**

4.1 PERFORMANCE REQUIREMENTS 10
4.2 SAFETY AND SECURITY REQUIREMENTS 10
4.3 SOFTWARE QUALITY ATTRIBUTES 10

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Ram Sundar Radhakrishnan	Created initial draft of the specifications.	08/12/2025
1.1	Ram Sundar Radhakrishnan	Created final draft of the specifications.	16/12/2025

1 Introduction

1.1 Document Purpose

The aim of this Software Requirements Specification is to collate all information related to, then give a detailed insight on the DAO-Based Ride Sharing Application, version 1.1.
The document also provides details on the capabilities required by stakeholders and their needs while also detailing the high-level product features.

1.2 Product Scope

The DAO-Based Ride Sharing Application is intended to connect commuters with space in their vehicles with passengers who need affordable and safe ways to commute. It also aims to improve popularity of ride-sharing by providing a reputation-based system to automatically grade passengers and hosts on their safe attitudes and behaviours.
The application uses Colony DAO platform for smart-contract based reputation management, passenger-host connection and dispute resolution. Payments will be settled off-chain using fiat currency to improve accessibility and retain maximum compliance with laws.
Colony DAO features a "reputation" system that allows for assigning influence to participants based on their existing work and historical behaviour.
The decentralized nature of the application allows for democratic dispute resolution and prevents any takeover of the community by any single force.

1.3 Intended Audience and Document Overview

The rest of this SRS provides a general description, including characteristics of the users of this project, the project's software, and the functional and data requirements of the product
Section 2 provides a general overview of the product.
Section 3 provides specific requirements like hardware and software interfaces.
Section 4 provides other non-specific requirements like performance, safety and security.

1.4 Definitions, Acronyms and Abbreviations

- BCT – Blockchain Technology
- DAO – Decentralized Autonomous Organization
- IOT – Internet Of Things
- KYC – Know Your Customer

1.5 Document Conventions

In general this document follows the IEEE formatting requirements.

1.6 References and Acknowledgments

- Rea, A., Kronovet, D., Fischer, A., Du Rose, J., & Organizations, for the Internet. (2018). COLONY.

- Han, J., Lee, J., & Li, T. (2025). A review of DAO governance: Recent literature and emerging trends. Journal of Corporate Finance, 102734.
- Morrison, R., Mazey, N. C., & Wingreen, S. C. (2020). The DAO controversy: the case for a new species of corporate governance?. Frontiers in Blockchain, 3, 25.
- Jaydarifard, S., Yigitcanlar, T., & Paz, A. (2025). Risk factors and safety strategies for mitigating violations, harassment and assault in taxi and ride-hailing services. Transport Reviews, 1-25.

2 Overall Description

2.1 Product Overview

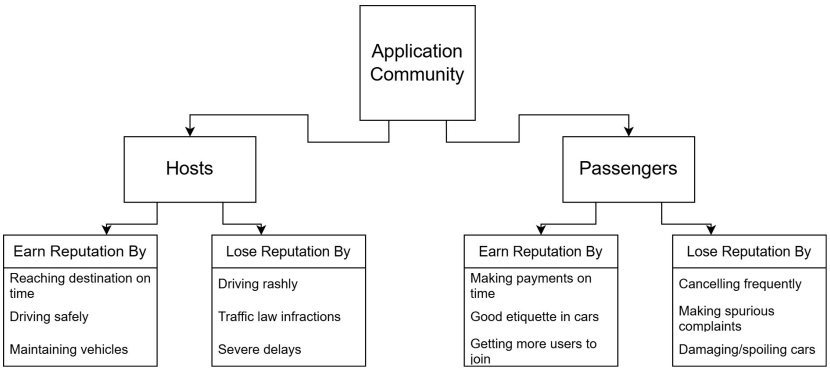
In today's scenario, ride-sharing and carpooling are used and mandated by governmental and non-governmental organizations as a way of reducing pollution and congestion. Commuters too are seeking to adopt such techniques to reduce the cost of commuting and also to generate some extra revenue from their own commutes.
A stumbling block in more widespread adoption of ridesharing is the perceived lack of safety, which has been exacerbated by the safety incidents including accidents, misbehaving passengers and drivers and a lack of transparency regarding the maintenance of standards for hosts and passengers.
Existing products are all centralized, with a central company (like Uber, OLA, Rapido, etc) controlling the platform, and setting the standards. A repeated complaint of users has been "shifting goalposts", or the change in standards that the company (central authority) requires of the users. Another complaint has been ineffective dispute resolution – with customer service agents replaced by AI, or trained to follow basic steps regardless of the actual situation.
By switching to a decentralized system, the following benefits are immediately realised:

- Hegemony of a single authority is avoided.
- Transparency of actions is guaranteed.

Furthermore, a decentralized system allows for immediate, fact-based conflict resolution by the use of smart-contracts and token staking.
The use of DAO platform Colony will add the following advantages:

- Reputation management
- Voting rights management
- Community management

Reputation is the main breakthrough offered by Colony DAO.
Users earn "Reputation" tokens based on their actions – actions that benefit the community earn a user reputation, while actions that harm a community cost a user their reputation.
Reputation cannot be traded between users, thus preventing any scamming by dummy accounts, and reputation also decays with time – encouraging active participation in the DAO and active usage of the platform.
With reputation, hosts and passengers can be assured of the other's good intentions, and each ride will see hosts and passengers staking their reputations for a good outcome of the ride – providing a strong economic and social incentive for good behaviours.



2.2 Product Functionality

The major functions of the system include:

- Reputation Management
- Voting on rule proposals via reputation-weighted community consensus
- Host-passenger matching off-chain
- Dispute resolution via smart contracts and reputation-based voting
- Off-chain payment management

2.3 Design and Implementation Constraints

DAOs remain a highly contentious technology, with many proponents against and for the usage of distributed systems.
By avoiding the use of any app-coins or crypto-coins for payment, or staking, or trading, the product avoids the maelstrom of regulatory issues with crypto currency.
Challenges remain in user on-boarding, oracle verification of off-chain events and maintaining community enthusiasm.
Furthermore, regulatory issues remain with regulation of decentralized applications.

2.4 Assumptions and Dependencies

2.4.1 Assumptions:

- Users possess smartphones/internet enabled devices with reliable connectivity.
- Users will be willing to undergo identity verification (KYC) as needed.
- Colony's smart-contract infrastructure remains stable and operational.
- Reputation data stored on the Colony network is considered accurate and tamper resistant.
- Off-chain fiat payment remains available and legally permitted.
- The decentralized model will be acceptable to regulators and not conflict with local transport or commerce laws.
- Users understand and accept the idea of staking reputation for ride commitments.
- Arbitrators with high reputation will act in good faith.

2.4.2 Dependencies:

- Colony Network (Smart Contracts and Reputation Engine)
- Blockchain Infrastructure (Ethereum L2 or compatible chain)
- Off-chain matching engine and backend services
- External Payment Gateways
- Mapping and Geolocation APIs (OpenStreetMap, Google Maps or similar)
- Mobile OS and device compatibility
- Identity Verification Services

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

3.1.1.1 User Onboarding Interfaces

- The onboarding screen will allow new users to create new accounts using phone number, email or both.
- The system will provide a structured workflow for optional identity verification (driver's license, ID proof).
- The interface will abstract all blockchain interactions – users will not be required to manage wallets or keys.
- Users will be able to select their role during onboarding – host, passenger, or both
- The system will display a brief explanation of the Reputation system and how it affects matching and trust. This explanation page will be available at any time the user requests it.
- The interface will allow hosts to submit vehicle details and required documentation.

3.1.1.2 Passenger Onboarding Interfaces

- The home screen will include a map with user location detected via GPS.
- The interface will include input fields for pickup location, drop locations and required vehicle type.
- The ride request screen will display status updates (Searching → Host Assigned → Arriving → Ongoing → Completed → Post-ride Feedback)
- A payment screen will support fiat payment methods – UPI, cash, credit/debit cards.
- The interface will then display the post-ride feedback options.
- The interface will then show the post-ride reputation resolution.
- A profile screen will display the passenger's recent transactions, rides, and reputation-related events.

3.1.1.3 Host Onboarding Interfaces

- The host interface shall allow toggling between Online and Offline status.
- The system shall display incoming ride requests along with passenger reputation and basic profile information.
- The interface shall allow the host to accept or decline the request; accepting shall stake a small amount of reputation.
- A navigation screen shall provide routing assistance to the passenger's pickup and drop locations.
- Hosts shall be able to rate passengers at the end of each ride.
- A performance dashboard shall show:
    - Host reputation score
    - Ride count
    - Cancellation penalties
    - Weekly performance summary
- Hosts shall have a vehicle management screen to upload documents and view status of verifications.

3.1.1.4 Reputation and DAO Interaction Interface

- The interface shall present a simplified view of the user's reputation score.
- The system shall display reputation broken down by Colony domains (Host, Passenger, Dispute Resolution).

- The system shall allow users to view their historical reputation changes with timestamps and reason codes.
- The interface shall display reputation staking events (e.g., accepting rides).
- Users shall be able to view DAO governance rules and decisions derived from Colony.
- Voting interfaces (if implemented) shall allow users with sufficient reputation to participate in community governance decisions.

3.1.1.5 Dispute and Arbitration Interface

3.1.1.5.1 For Hosts and Passengers

- The interface shall provide an option to file a dispute after ride completion.
- Users shall be able to submit a description, select dispute category, and upload evidence (photos, logs, etc.).
- The system shall show dispute status (Submitted → Under Review → Resolved).
- The decision screen shall show arbitration results and reputation adjustments.

3.1.1.5.2 For Arbitrators

- An arbitrator dashboard shall show unresolved disputes requiring attention.
- The interface shall display all evidence submitted for each case.
- Arbitrators shall cast votes or select from predefined decision options.
- The interface shall show any reputation gained or lost as a result of arbitration performance.

3.1.1.6 Administrative Interface (for DAO Moderators, Compliance Team)

- The system shall provide a dashboard summarizing global platform metrics (ride volume, disputes, average reputation).
- Moderators shall have access to flagged disputes requiring manual oversight.
- The interface shall allow review and approval of host verification documents.
- The system shall permit modification of platform-wide rule constants (penalties, thresholds), subject to governance approval.
- The interface shall log all administrative actions for auditing.

3.1.2 Hardware Interfaces

In this project, the hardware interfaces refer to the physical oracles used by the DAO to interact with real world “sources of truth”.

- Smartphone embedded sensors – GPS, Accelerometer, Gyroscope
- Camera
- RFID Tags

3.1.3 Software Interfaces

These are the software subsystems that this project interacts with.

- Ethereum L2/Blockchain Interface
- Colony SDK
- Backend Application Layer
- Payment Gateway Interfaces
- Geolocation and Mapping APIs
- Notification Services
- Identity Verification Services

3.2 Functional Requirements

3.2.1 FR1 The system shall allow users to register, authenticate, choose their role (Passenger/Host), and optionally complete identity verification.

3.2.2 FR2 The system shall allow hosts to upload and manage vehicle and license documents for approval.

3.2.3 FR3 The system shall allow passengers to input pickup/drop locations, compute estimates, and submit ride requests.

3.2.4 FR4 The system shall allow hosts to toggle availability, receive ride requests, and accept or decline them.

3.2.5 FR5 The system shall obtain and process GPS data to track host and passenger locations throughout the ride lifecycle.

3.2.6 FR6 The system shall record and update reputation values for passengers and hosts on the Colony network based on actions, penalties, and ratings.

3.2.7 FR7 The system shall match passengers to hosts using a scoring formula that includes reputation, distance, and past reliability.

3.2.8 FR8 The system shall support off-chain fiat payments (UPI, card, cash) and store payment confirmations for audit and dispute purposes.

3.2.9 FR9 The system shall allow users to file disputes, upload evidence, and track dispute status.

3.2.10 FR10 The system shall assign disputes to eligible arbitrators, record verdicts, update arbitration-domain reputation, and notify involved users.

3.3 Use Case Model

3.3.1 Use Case #1 – U1: Submit Ride Request

Author – Ram Sundar Radhakrishnan

Purpose - To allow a passenger to request a ride by providing pickup and drop locations, triggering the system's matching engine.

Requirements Traceability –

- FR3 Ride Request Handling

- FR5 Real-Time Location Tracking
- FR7 Matching Engine
- NFR Performance, Reliability, Usability

Priority – **High.** This is a core function; the system cannot operate without successful ride requests.

Preconditions –

- User is registered and authenticated.
- GPS location is enabled and available.
- Passenger role is active (not suspended/frozen).

Post conditions –

- A ride request is created and stored.
- Matching engine starts processing the request.
- Passenger receives confirmation of host assignment.

Actors – Passenger (primary), Matching Engine (system actor), GPS hardware (device actor)

Extends – None

Flow of Events

1. Basic Flow - flow of events normally executed in the use-case
    - The passenger opens the app and selects "Request Ride."
    - The system retrieves the passenger's current GPS coordinates.
    - The passenger enters the drop-off location.
    - The system calculates estimated fare and time.
    - The passenger confirms the ride request.
    - The system creates the ride request and sends it to the matching engine.
    - The system notifies the passenger once a host is assigned.
2. Alternative Flow - a secondary flow of events due to infrequent conditions

    Passenger chooses manual pickup location:

    - Passenger taps "Edit Pickup."
    - Passenger selects a different pickup point on the map.
    - System recalculates estimates before submitting request.
3. Exceptions - Exceptions that may happen during the execution of the use case

E1: GPS unavailable: System displays "Unable to fetch location" and cancels the request.

E2: Network failure: System retries 3 times; if unsuccessful, displays error message.

E3: No hosts available: System notifies passenger: "No hosts in your area currently."

Includes none

Notes/Issues - Need to clarify whether ride requests should be rate-limited to prevent spam.

4 Other Non-functional Requirements

4.1 Performance Requirements

PR1- The system shall complete ride matching operations within 2 seconds under normal operating conditions.

PR2- GPS location updates shall be processed at a minimum frequency of once every 5 seconds.
PR3- The system shall support at least 150 concurrent users without degradation of core services.
PR4- Retrieval of reputation values (on-chain or cached) shall take no more than 1 second per request.
PR5- The system's backend services shall maintain an average response time of < 300 ms for all critical API endpoints.

4.2 Safety and Security Requirements

SSR1- All data transmitted between client and server shall be encrypted using TLS/HTTPS.
SSR2- User authentication shall use OTP or MFA, preventing unauthorized access.
SSR3- Sensitive data (payment information, blockchain keys) shall not be stored on-device or server in raw form.
SSR4- Access to dispute evidence and arbitration data shall be restricted to authorized arbitrators only.
SSR5- The system shall provide an SOS safety feature allowing users to contact emergency services during a ride.

4.3 Software Quality Attributes

SQA1- The system shall maintain 99% uptime for core services including authentication, ride requests, and matching.
SQA2- The UI shall be intuitive, mobile-friendly, and require no blockchain knowledge, displaying reputation in human-readable form.
SQA3- The system shall follow a modular architecture so that the matching logic, DAO parameters, or reputation rules can be updated without major redesign.
SQA4- The system shall support horizontal scaling of backend components to accommodate increased ride volume and user growth.
SQA5- The application shall run on Android and iOS, using standard APIs for GPS, mapping, and notifications.